- You no longer have the same guarantees you had when you, for instance, called a method on a `Conversation` object to add a `Message`. Before, you were guaranteed that `addMessage` would never fail since it was a simple operation on an in-memory list (barring the exceptional case of the JVM running out of memory). Now, the database might return an error at any `addMessage` call. The insert might fail, or the database might not be available because the database server crashed or due to a network problem.
- The in-memory version had a sprinkling of locks to ensure that the data wouldn't get corrupted by concurrent users. Now that you're using "Database X," you have to handle that problem and make sure you don't end up with duplicate records or other inconsistent data. You need to find out how to do that with the Database X library. All simple method calls to objects effectively become database operations, some of which must work in concert. Starting a conversation, for instance, requires at least an insert of a row in the `Conversation` and `Message` tables.
- The in-memory version was easy to test, and unit tests ran fast. Now you run Database X locally for the tests and add database test utilities to isolate tests. Unit tests run a lot slower. But you tell yourself, "At least I'm also testing the Database X operations," which were not as intuitive as you expected—they're very different from previous databases you've worked with.

You run into performance problems when you're porting the in-memory code directly to database calls since every call now has network overhead. So, to optimize query performance, you design specific database structures that are specific to your choice of database (SQL or NoSQL, it doesn't matter). The objects are anemic